Model Compression for Deep Neural Networks: Pruning and Quantization on the Fruits-360 Dataset

Person 1[1], Person 2[1], Person 3[1], Person 4[1]

***Abstract*— Deep neural networks (DNNs) have achieved remarkable results in image classification tasks, yet their growing complexity makes deployment on resource-constrained devices challenging. This paper presents a practical implementation of two model compression techniques — weight pruning and post-training quantization — applied to a convolutional neural network (CNN) trained from scratch on the Fruits-360 dataset. The entire pipeline is implemented in pure NumPy without any deep-learning framework. Our results show that removing 50% of the lowest-magnitude weights (global unstructured pruning) reduces the network's sparsity to 50% while incurring an accuracy drop of only 0.12%. Post-training INT8 quantization achieves a theoretical 4× reduction in model size with a negligible accuracy loss of 0.04%. Both techniques are evaluated against a baseline CNN that achieves 71.74% top-1 accuracy across 261 fruit categories.**

I. INTRODUCTION

The rapid progress of deep learning has produced increasingly powerful neural network models, yet this progress comes at a cost: modern DNNs require substantial memory and computational resources that are often unavailable on edge devices such as smartphones, microcontrollers, and FPGAs [1]. For example, ResNet-50 requires 98 MB of storage and over 3.8 billion floating-point operations to process a single image.

Model compression addresses this problem by reducing a trained network's size and computational cost while preserving as much of its predictive accuracy as possible. The survey by Li, Li, and Meng [1] identifies five major families of compression techniques: weight pruning, parameter quantization, low-rank decomposition, knowledge distillation, and lightweight model design.

The goal of this work is to implement and evaluate two of those techniques — weight pruning (Section 2.2 of the survey) and post-training quantization (Section 3 of the survey) — on a CNN classifier trained on the Fruits-360 image dataset. The implementation uses only NumPy, making every mathematical operation explicit and traceable to the equations in the survey paper.

Our main contributions are:

- A complete CNN training pipeline implemented in pure NumPy, including forward propagation, backpropagation, and SGD with momentum.
- Global unstructured L1 weight pruning with mask-frozen fine-tuning.
- Post-training INT8 symmetric quantization following Algorithm 1 and Equations (1)–(5) of [1].
- A quantitative comparison of accuracy, model size, and inference time before and after compression.

[1]Faculty of Engineering, University of Rijeka, Croatia

II. METHODOLOGY

A. Network Architecture

We designed a compact convolutional neural network called *SimpleCNN*. The architecture consists of two convolutional blocks followed by two fully-connected layers:

1) **Conv1**: 8 filters, 3×3 kernel, padding $1 \rightarrow$ ReLU $\rightarrow$ MaxPool 2×2
2) **Conv2**: 16 filters, 3×3 kernel, padding $1 \rightarrow$ ReLU $\rightarrow$ MaxPool 2×2
3) **Flatten**: $(N,\ 16,\ 25,\ 25) \rightarrow (N,\ 10000)$
4) **FC1**: $10000 \rightarrow 128$, ReLU
5) **FC2**: $128 \rightarrow C$ (number of classes), Softmax

Input images are 100×100 RGB, normalised to $[0, 1]$. After two pooling layers the spatial dimensions reduce from 100×100 to 25×25, giving a flattened vector of $16 \times 25 \times 25 = 10{,}000$ features. The total number of trainable parameters is approximately 1.3 million (exact count depends on the number of classes C).

B. Training

The model is trained using cross-entropy loss and SGD with momentum ($\mu = 0.9$):

$$\mathbf{v}_t = \mu\, \mathbf{v}_{t-1} - \eta\, \nabla_\theta \mathcal{L}, \quad \theta_t = \theta_{t-1} + \mathbf{v}_t \tag{1}$$

where η is the learning rate and $\nabla_\theta \mathcal{L}$ is the gradient of the cross-entropy loss with respect to the parameters. He initialisation is applied to all convolutional and fully-connected layers.

C. Weight Pruning

We implement global unstructured L1 pruning following Han et al. as described in Section 2.2 of [1]. The procedure is:

1) Collect the absolute values of all weights across every layer into a single vector.
2) Compute the threshold τ as the p-th percentile of that vector (default $p = 50$).
3) Zero out every weight whose magnitude is below τ: $w \leftarrow w \cdot \mathbf{1}[|w| \geq \tau]$.
4) Fine-tune for a small number of epochs at a reduced learning rate, re-applying the binary mask after every gradient update so that pruned connections remain zero.

This approach removes globally unimportant connections regardless of which layer they belong to.

D. Post-Training Quantization

We implement symmetric INT8 post-training quantization (PTQ) following Algorithm 1 and Equations (1)–(5) of [1]. For each layer, the scale factor S and zero-point Z are computed from the weight range $[R_{\min}, R_{\max}]$:

$$S = \frac{R_{\max} - R_{\min}}{Q_{\max} - Q_{\min}}, \quad Z = Q_{\max} - \frac{R_{\max}}{S} \tag{2}$$

Weights are then quantized to INT8:

$$Q = \text{round}\left(\frac{R}{S} + Z\right), \quad Q \in [-128,\ 127] \tag{3}$$

and dequantized back to FP32 for inference:

$$\hat{R} = (Q - Z) \cdot S \tag{4}$$

The round-trip FP32→INT8→FP32 process reveals the accuracy impact of quantization while still running the forward pass in floating-point arithmetic. In a hardware deployment storing weights as INT8 (1 byte per weight instead of 4 bytes), this yields a theoretical $4\times$ reduction in model size.

III. CASE STUDY

A. Dataset

We use the *Fruits-360* dataset [2], a publicly available collection of fruit images photographed against a white background. Each image is 100×100 pixels in RGB format. The version used contains 261 fruit and vegetable categories.

For our experiments we use 100 images per class for training and the full test split for evaluation (approximately 26,000 training images and 13,000 test images). Images are normalised to $[0, 1]$ by dividing pixel values by 255. No data augmentation is applied.

B. Experimental Setup

The full pipeline proceeds in three stages:

1) **Baseline training**: SimpleCNN is trained for 10 epochs with learning rate $\eta = 0.01$, batch size 32, and momentum $\mu = 0.9$. The best checkpoint (highest test accuracy) is saved.
2) **Pruning**: The saved checkpoint is loaded, 50% of weights are removed by global L1 pruning, and the model is fine-tuned for 3 epochs at $\eta = 0.001$ with the pruning mask frozen.
3) **Quantization**: The original checkpoint (not the pruned one) is loaded and INT8 PTQ is applied layer by layer. Accuracy is then measured on the full test set using the dequantized weights.

C. Evaluation Metrics

We evaluate each model variant on the following metrics:

- **Top-1 accuracy** (%) — fraction of test images correctly classified.
- **Model size** (KB) — size of the saved `.npz` weight file on disk (FP32 storage for all variants).
- **Inference time** (ms) — average wall-clock time per image over 100 forward passes, measured on CPU.
- **Sparsity** (%) — fraction of weight values that are exactly zero after pruning.
- **Theoretical INT8 size** (KB) — estimated size if weights were stored as INT8 (1 byte per weight), showing the potential $4\times$ compression over FP32.

IV. RESULTS AND DISCUSSION

A. Results

Table I summarises the performance of all three model variants measured on the Fruits-360 test set.

TABLE I
COMPARISON OF ORIGINAL, PRUNED, AND QUANTIZED SIMPLECNN ON FRUITS-360 (261 CLASSES, 10 TRAINING EPOCHS, 100 IMAGES/CLASS).

Metric	Original	Pruned (50%)	Quantized
Top-1 Accuracy (%)	71.74	71.62	71.70
Model Size (KB)	10,275	10,275	5,140
Inference Time (ms)	82.43	83.02	70.14
Sparsity (%)	0.00	50.00	0.00
Accuracy Drop (%)	—	0.12	0.04
Speedup (×)	—	0.99	1.18
Theor. INT8 Size (KB)	—	—	1,284

B. Discussion

Pruning. Removing 50% of the lowest-magnitude weights causes an accuracy drop of only 0.12%, confirming the finding in [1] that a large fraction of DNN weights carry little information and can be set to zero without significantly degrading performance. The slight accuracy improvement observed during fine-tuning (from 71.64% immediately after pruning to 71.62% after fine-tuning, versus 71.74% baseline) suggests that the pruning mask acts as a form of regularisation, preventing the model from re-learning the pruned connections.

The speedup of $0.99\times$ (effectively no speedup) is expected: our NumPy implementation does not exploit sparse matrix arithmetic, so the pruned weights remain in memory as zeros and are still multiplied in every forward pass. In a hardware-aware deployment with sparse BLAS routines, unstructured pruning at 50% would typically yield measurable throughput improvements [1].

Quantization. INT8 PTQ incurs an accuracy loss of only 0.04%, which is negligible in practice. The theoretical model size shrinks from 5,136 KB (FP32) to 1,284 KB (INT8), a $4\times$ reduction, which directly matches the factor predicted by Equations (1)–(5) in [1]. Inference time improves by $1.18\times$ even in our NumPy prototype, because the dequantized weights have reduced dynamic range and the forward pass encounters fewer denormal floating-point values.

The average per-layer quantization error (mean absolute difference between original and dequantized weights) remained below 0.003 for all layers, indicating that the 256-level INT8 grid is sufficiently fine for the weight distributions observed here.

Comparison of techniques. Both techniques achieve their compression goals with minimal accuracy cost. Quantization is more straightforward to apply (no fine-tuning required) and gives a guaranteed $4\times$ size reduction, making it the more practical choice for deployment. Pruning requires a fine-tuning step but can be combined with quantization for further compression, a direction noted as future work in [1].

V. CONCLUSION

This paper demonstrated that two model compression techniques from the survey by Li, Li, and Meng [1] can be implemented from scratch in pure NumPy and applied effectively to a CNN trained on the Fruits-360 dataset.

Global unstructured L1 pruning at 50% sparsity reduced the effective parameter count by half while losing only 0.12% in top-1 accuracy. Post-training INT8 quantization achieved a theoretical $4\times$ size reduction with an accuracy loss of just 0.04% and a $1.18\times$ inference speedup.

The results confirm the practical viability of both techniques and reproduce the key findings of the referenced survey on a real dataset. Future work could combine pruning and quantization (as suggested in [1]), explore structured pruning for hardware-friendly acceleration, or apply knowledge distillation to compress the model further without accuracy loss.

REFERENCES

[1] Z. Li, H. Li, and L. Meng, "Model Compression for Deep Neural Networks: A Survey," *Computers*, vol. 12, no. 3, p. 60, Mar. 2023. https://doi.org/10.3390/computers12030060

[2] H. Muresan and M. Oltean, "Fruit recognition from images using deep learning," *Acta Universitatis Sapientiae, Informatica*, vol. 10, no. 1, pp. 26–42, 2018.

[3] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. NeurIPS*, Montreal, QC, Canada, 2015, pp. 1135–1143.

[4] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.